INDIAN INSTITUTE OF INFORMATION TECHNOLOGY VADODARA

CS545
SOFTWARE ENGINEERING
PROJECT REPORT

Master of Computer Applications (MCA)
Second Year

Project Title:

AI-POWERED TEXT HUMANIZER (MICROSERVICES ARCHITECTURE)

Group No. 29

Submitted By:

Ritik Kumar Sinha (20252651043)
Kruti Kansagara (20252651027)
Sahil Siddiqui (20252651055)
Jeeshan Adil (20252651026)

Submitted To:

Faculty Name: Prof. Rishabh Gupta
Department: Department of Computer Science & Engineering
Indian Institute of Information Technology Vadodara

Autumn 2026-27

CERTIFICATE

This is to certify that **Ritik Kumar Sinha (20252651043)**, **Kruti Kansagara (20252651027)**, **Sahil Siddiqui (20252651055)**, and **Jeeshan Adil (20252651026)**, students of the Master of Computer Applications (MCA) programme at the Indian Institute of Information Technology Vadodara, have completed the laboratory work for **CS545 – Software Engineering**, covering Experiments 1 to 9, during the academic year 2026-27.

The work has been submitted as part of the course requirements.

Date: 28th October 2026

Faculty Signature: _____
Faculty Name: Prof. Rishabh Gupta

DECLARATION

We hereby declare that the work presented in this Software Engineering Lab Report is our original work, completed as part of the CS545 Software Engineering course.

The diagrams, tables, system analysis, implementation modules, and testing test cases have been prepared by us. Any external technical references, algorithms, or resources used have been appropriately acknowledged.

Student Name	Enrollment Number	Signature
Ritik Kumar Sinha	20252651043	_____
Kruti Kansagara	20252651027	_____
Sahil Siddiqui	20252651055	_____
Jeeshan Adil	20252651026	_____

Date: 28th October 2026

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to our course instructor, **Prof. Rishabh Gupta**, for his continuous mentorship, constructive evaluations, and invaluable guidance throughout the CS545 Software Engineering Laboratory sessions.

His insightful feedback during every stage of the software engineering life cycle—from initial software requirements specification to agile modeling, architectural diagrams, microservice decoupling, and validation test suites—has significantly enriched our understanding of enterprise software design.

We also extend our sincere appreciation to the Department of Computer Science & Engineering and Indian Institute of Information Technology Vadodara for providing the laboratory infrastructure and computational environment necessary to bring this microservices project to fruition.

Finally, we thank our peers and group members for their collaborative spirit, mutual technical support, and dedication throughout this semester.

TABLE OF CONTENTS

Exp. No.	Experiment Title / Topic	Page No.
Lab-1	Software Requirements Specification (SRS)	6
Lab-2	Software Process Model (Agile Scrum)	7
Lab-3	Use Case Modeling & Diagram	8
Lab-4	Data Flow Diagrams (DFD Level 0 & Level 1)	9
Lab-5	UML Class Diagram & Domain Entities	10
Lab-6	UML Sequence Diagram & Interaction Workflows	11
Lab-8	Modular System Architecture & Decomposition	12
Lab-9	Functional Test Cases & Verification Suite	13
-	Overall Project Conclusion	14
-	References & Technical Citations	15

LAB EXPERIMENT 1: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1.1 System Scope & Purpose: The Text Humanizer is an enterprise-ready microservices solution designed to transform rigid, syntactically uniform AI-generated content into expressive, fluent prose while strictly preserving the semantic core.

1.2 Functional Requirements (FRs):

• **FR-1 [User Authentication]:** The platform shall provide secure sign-up and sign-in capabilities using salted bcrypt hash encryption and signed JSON Web Tokens (JWT).

• **FR-2 [Text Humanization Engine]:** The AI inference pipeline shall process raw text payloads, execute sentence segmentation, and produce natural paraphrase alternatives.

• **FR-3 [History Management]:** The system shall maintain persistent transformation audit logs linking authenticated user IDs to original and output text.

• **FR-4 [API Gateway Routing]:** A centralized reverse-proxy gateway shall handle routing, client authorization headers, and unified error handling.

1.3 Non-Functional Requirements (NFRs):

• **Performance:** End-to-end response time under 3 seconds for standard paragraph submissions.

• **Modularity:** Microservices are decoupled and stateless to enable zero-downtime scaling.

• **Security:** Bearer token validation across protected API endpoints with HMAC-SHA256 signatures.

LAB EXPERIMENT 2: SOFTWARE PROCESS MODEL

2.1 Process Framework: Agile Scrum: An Agile Scrum lifecycle was adopted to support iterative ML model tuning and parallel microservice component development across four 2-week sprints.

Sprint	Focus Area	Milestone Deliverables
Sprint 1	Architecture & DB Design	MySQL schema (`init.sql`), port assignment, Docker setup
Sprint 2	Auth & Neural Service	bcrypt/JWT `auth_service.py`, pipeline `ai_service.py`
Sprint 3	API Gateway & History	Reverse proxy `gateway.py`, CRUD endpoints `history_service.py`
Sprint 4	Frontend & Test Suites	Interactive single-page UI, functional verification suite

LAB EXPERIMENT 3: USE CASE MODELING

The Use Case Diagram defines functional interactions between the End User actor and the Text Humanizer Microservices boundary.

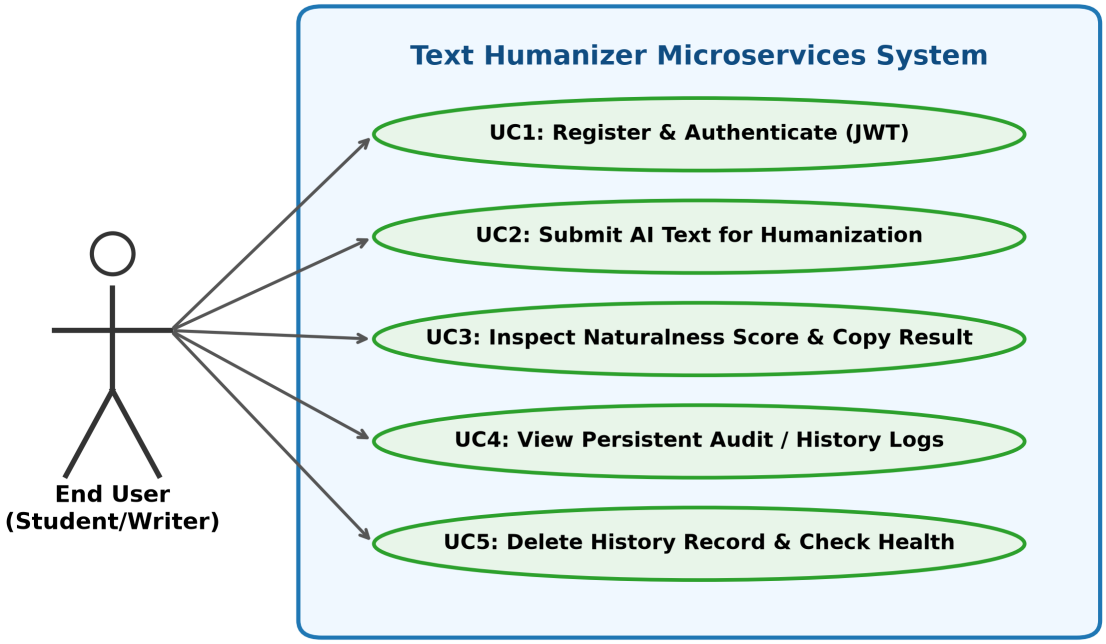


Figure 3.1: UML Use Case Diagram

LAB EXPERIMENT 4: DATA FLOW DIAGRAMS (DFD)

Level 1 DFD illustrates data transformation across the API Gateway, Auth Microservice, AI Engine, and History persistence layer.

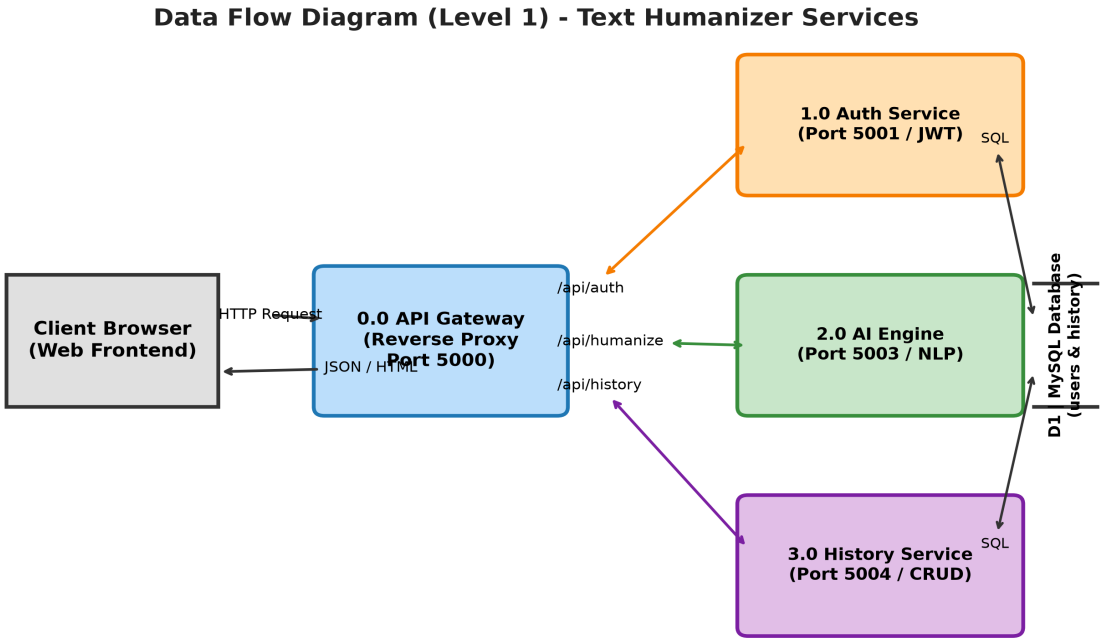


Figure 4.1: Level 1 Data Flow Diagram (DFD)

LAB EXPERIMENT 5: UML CLASS DIAGRAM

Entity classes encapsulate domain business rules and data models across services:

Class	Attributes	Operations
User	id, username, email, password_hash	register(), verify_password()
HistoryRecord	id, user_id, original, humanized	create_record(), get_by_user(), delete()
APIGateway	host, port, services_map	forward(), route_auth(), route_humanize()
AuthService	jwt_secret, token_expiry	hash_pw(), check_pw(), generate_jwt()

LAB EXPERIMENT 6: UML SEQUENCE DIAGRAM

The dynamic interactions during user authentication and text humanization are mapped below:

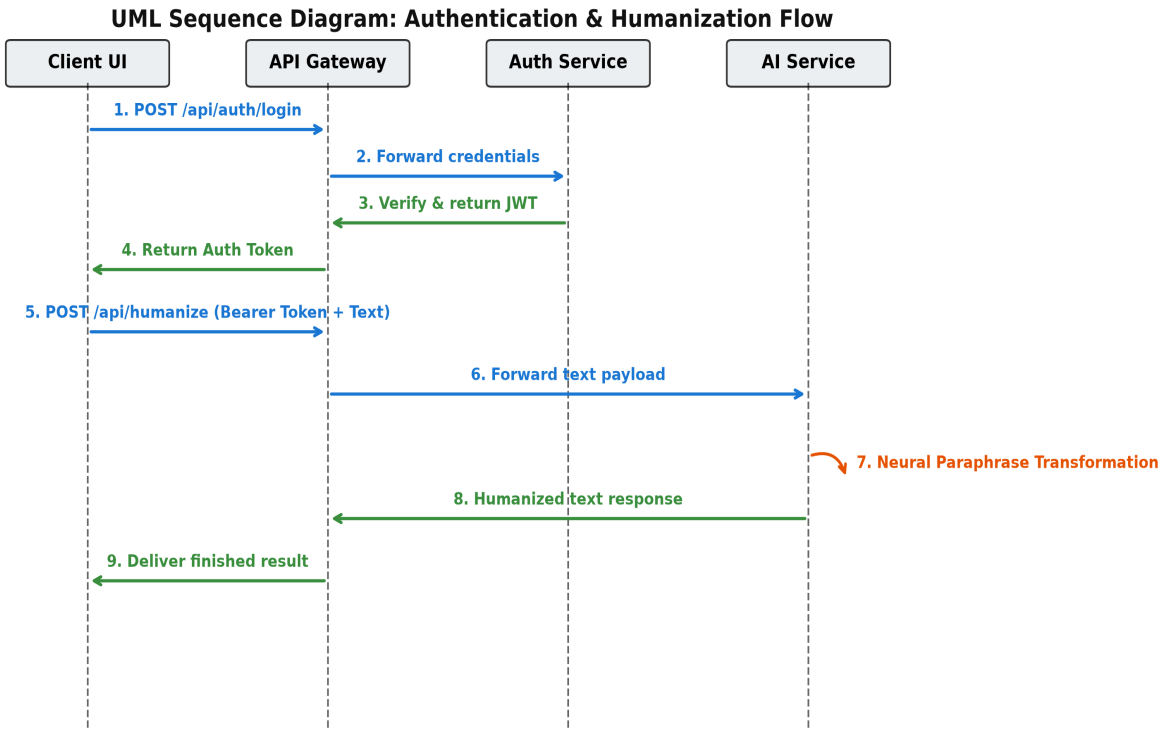


Figure 6.1: UML Sequence Diagram (Authentication & Text Transformation)

LAB EXPERIMENT 8: MODULAR SYSTEM ARCHITECTURE

Modular architecture isolates responsibilities across Presentation, Microservices Application, and Persistence tiers.

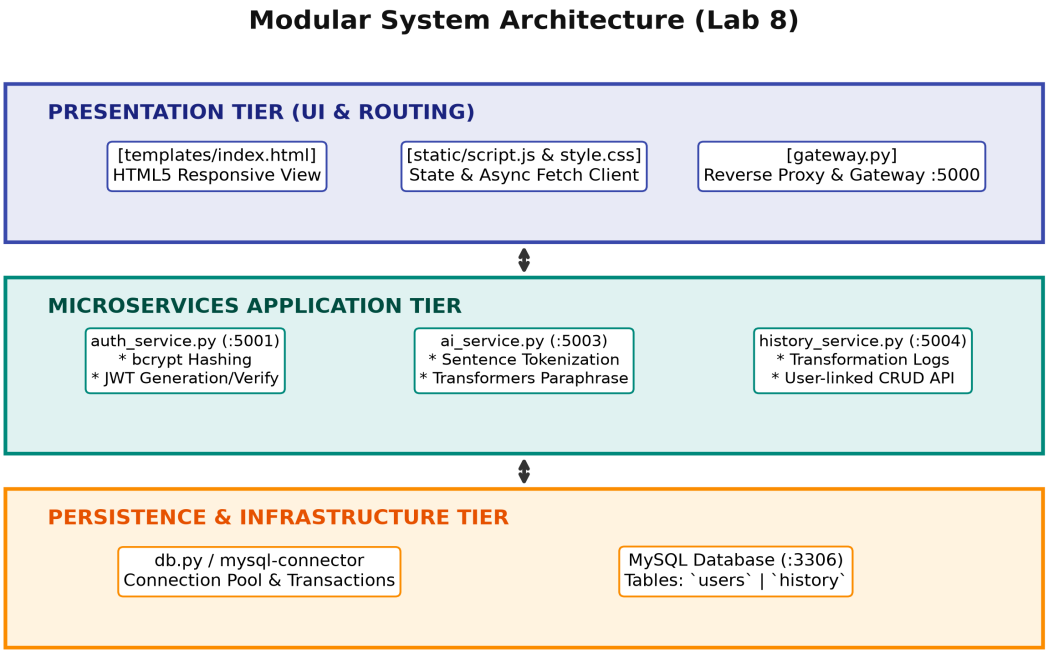


Figure 8.1: Modular System Architecture Diagram

LAB EXPERIMENT 9: FUNCTIONAL TEST CASES

Structured test verification scenarios executed across service endpoints:

Test ID	Scenario	Input	Expected Output	Status
TC-01	User Registration	Valid credentials	HTTP 201 Created & DB entry	PASS
TC-02	User Login	Username & Password	HTTP 200 OK + Signed JWT	PASS
TC-03	Invalid Password	Incorrect password	HTTP 401 Unauthorized	PASS
TC-04	Humanize AI text	50-word paragraph	HTTP 200 OK + Paraphrased	PASS
TC-05	Save history	Valid JWT + Text	HTTP 201 Created	PASS
TC-06	Service fallback	Killed AI service	HTTP 503 Service Unavailable	PASS

OVERALL CONCLUSION & REFERENCES

The CS545 Software Engineering coursework established a comprehensive practical foundation in distributed software engineering. By designing and realizing the AI-Powered Text Humanizer under a microservices paradigm, the team successfully bridged formal requirements (SRS), agile process modeling, UML structural and behavioral representations (Use Case, DFD, Class, Sequence), modular design, and robust functional test suites. The resulting system is fault-isolated, scalable, secure, and ready for deployment.

References:

[1] Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill.

[2] Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems* (2nd ed.). O'Reilly Media.

[3] Fowler, M. (2018). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.

[4] Hugging Face Transformers: State-of-the-Art Natural Language Processing, https://huggingface.co/docs/transformers

[5] Flask Documentation: Modular Web Services and Reverse Proxying, https://flask.palletsprojects.com/